

pyenlight

Developer's Guide

Alexandros Aslanidis

Department of Informatics & Telematics, Harokopio University of Athens



Table of Contents

Table of Contents	2
1. Scope and Purpose	4
2. System Architecture	4
3. End-to-End Data Flow	4
4. Repository and Module Map	5
5. Configuration and Precedence	5
6. Geometry and Array Conventions	5
7. Physical-Layer Models	5
8. MAC-Layer Models	6
9. Energy and Harvesting Models	6
10. Persistence and Reproducibility	6
11. Minimal Execution Example	6
12. Complete API Reference	7
12.1 Module `channel.py`	7
Class `ChannelEngine`	7
12.2 Module `config.py`	7
Class `PhysicsConfig`	7
Class `SpectralConfig`	7
Class `EnvironmentConfig`	7
Class `HardwareConfig`	7
Class `CommConfig`	7
Class `DeviceConfig`	7
Class `EnLightConfig`	7
12.3 Module `devices.py`	8
Class `TIA`	8
Class `IRdriver`	8
Class `PV`	8
Module-level functions	9
12.4 Module `energy.py`	9
Class `EnergyManager`	9
12.5 Module `geometry.py`	9
Class `Elements`	9
Class `OpticalRxElements`	9
Class `OpticalTxElements`	9
Class `RFTxElements`	9
Class `Surface`	10
Class `RoomBuilder`	10
Class `Room`	10
12.6 Module `interface.py`	10

Class `PhyResultsDTO`	10
12.7 Module `mac.py`	10
Class `MAC_Params`	10
Class `SlotClock`	11
Class `VLC_Channel`	11
Class `NodeStats`	11
Class `VLC_Node`	11
Module-level functions	11
12.8 Module `nodes.py`	12
Class `NodeBuilder`	12
Class `SNManager`	12
Class `MNManager`	12
Class `ANManager`	12
12.9 Module `orchestrator.py`	12
Class `oPhyGains`	12
Class `PhyNet`	13
12.10 Module `physics.py`	13
Class `SpectralPhysics`	13
12.11 Module `utils.py`	13
Module-level functions	13
Known Limitations and Maintenance Priorities	14

1. Scope and Purpose

pyenlight is a cross-layer simulator for indoor optical and hybrid optical/RF IoT networks. It connects room geometry, optical and RF propagation, receiver hardware, spectral response, PHY reliability, CSMA/CA, BTMA, energy consumption, PV harvesting, and battery lifetime in one workflow.

This document describes mostly the software implementation. For the model description, check the corresponding paper.

2. System Architecture

The library is organised around two main execution objects: **PhyNet** and **EnergyManager**.

PhyNet receives the design dictionary and creates the configuration, room geometry, node builders, node managers, channel engine, and physical-layer gain model.

RoomBuilder and **Room** construct the room surfaces, windows, RIS elements, and blockers, while **NodeBuilder**, **SNManager**, **MNManager**, and **ANManager** create the sensor, master, and ambient-node objects.

oPhyGains uses **ChannelEngine** to calculate the downlink, uplink, sensor-to-sensor, ambient, LoS, diffuse, RIS, and RF channel components. **PhyNet** then calculates the receiver noise, bandwidths, received signals, SNRs, and BER related metrics, and, when **budget_run=True**, the required IR or RF Tx powers. The required PHY outputs are exported as a **PhyResultsDTO**.

EnergyManager receives this DTO and calculates the sensor-state durations and energy consumption. When **MAC=True**, it internally calls the MAC simulator through **_apply_mac_times()** and **call_MAC()**, replacing the nominal communication durations with the simulated CCA, backoff, Tx, waiting, and Rx times. Finally, it calculates the harvested energy, daily net energy, battery lifetime, and per-node result table.

3. End-to-End Data Flow

The simulation starts by creating a **PhyNet** object from the design dictionary and an **EnLightConfig** object, which contains physical constants and fallback values for design parameters that are not explicitly defined. During its initialisation, **PhyNet** constructs the room and node models, evaluates the optical and RF channels, calculates the received signals and noise, and derives the PHY metrics. When **budget_run=True**, it also adjusts the sensor node transmitter orientation (toward the master node receiver) and Tx power according to the link-budget calculation.

The PHY outputs required by the later layers are then exported using **export_energy_telemetry()**, which returns a **PhyResultsDTO**.

The DTO is passed to **EnergyManager**. When **MAC=False**, **EnergyManager** calculates the energy consumption using the nominal communication durations. When **MAC=True**, it internally executes the MAC simulator, replaces the nominal communication durations with the simulated per-node CCA, backoff, Tx, waiting, and Rx times, and then calculates the MAC-aware energy consumption.

Finally, **EnergyManager** calculates the harvested energy, daily energy balance, and idealised battery lifetime. The per-node results are returned as a Pandas DataFrame and can be exported to CSV.

4. Repository and Module Map

Module	Primary responsibility	Key types/functions
core/config.py	Dataclass defaults for physics, spectra, environment, hardware, communication, and devices.	EnLightConfig and six nested configuration dataclasses
core/utils.py	Help functions: Array normalization, blockage geometry, Q (and inverse Q) functions, grids, and alignment.	to_vec_Nx3, calculate_blockage_mask, Qfunction, align_to
core/interface.py	Generates the object to pass data from PHY to energy/MAC.	PhyResultsDTO
environment/geometry.py	Element batches, surfaces, room construction, and special surfaces.	Elements, Surface, RoomBuilder, Room
environment/channel.py	LoS, diffuse, RIS, and RF propagation.	ChannelEngine
hardware/physics.py	Spectra, filters, detector response, effective responsivity.	SpectralPhysics
hardware/devices.py	TIA, IR/RF Tx current, and PV small-signal receiver.	TIA, IRdriver, PV, RF_calc_I
network/nodes.py	Node dictionary parsing and sensor/master/ambient managers.	NodeBuilder, SNManager, MNManager, ANManager
network/orchestrator.py	Complete PHY execution and telemetry export.	oPhyGains, PhyNet
network/mac.py	SimPy CSMA/CA, BTMA, hidden nodes, and aggregation.	MAC_Params, VLC_Channel, VLC_Node, call_MAC
hardware/energy.py	Per-state energy, MAC-aware timings, harvesting, lifetime.	EnergyManager

5. Configuration and Precedence

EnLightConfig provides the default parameter values, while the design dictionary defines the scenario-specific geometry, nodes, and operating conditions.

The design dictionary is not formally schema validated. Therefore, key names and array dimensions must be checked carefully. Scalar values are automatically broadcast across nodes, which is convenient but may also conceal unintended use of a common value for all nodes.

6. Geometry and Array Conventions

Positions and orientation vectors use shape (N, 3). Scalar per-node values use shape (N,) or are broadcast from a scalar. Channel matrices are pairwise Tx-by-Rx arrays. Hidden-node matrices use directed indexing: `hidden_node_mask[transmitter, listener]` indicates that the listener cannot detect the transmitter.

Room boundaries are discretized into surface patches. Diffuse and RIS complexity therefore scales with the number of patches and reflection order, not only with the number of sensor nodes.

7. Physical-Layer Models

Optical LoS gain uses Lambertian emission, distance attenuation, transmitter and receiver angles, receiver area, field of view, and blockage. Diffuse gain propagates through room patches over the configured number of bounces. RIS gain evaluates a two-hop path through RIS elements. RF gain follows the configured indoor path-loss model.

SpectralPhysics converts optical power into detector current using effective responsivity obtained from the source spectrum, detector response, and filter transmission. PD receivers use TIA and shot-noise terms. PV receivers use a nonlinear DC operating point and a frequency-dependent small-signal model.

8. MAC-Layer Models

The MAC layer is an event-driven SimPy model supporting slotted and unslotted CSMA/CA, random initial offsets, periodic or stochastic traffic, finite backoff and retransmission limits, collisions, ACK reception, PHY failures, hidden nodes, and BTMA.

The model distinguishes channel-access failure from collision, uplink PHY loss, and ACK/downlink loss. This distinction matters because the corresponding state durations and energy costs differ.

9. Energy and Harvesting Models

EnergyManager constructs each sensor-node cycle from the initialization, sensing, processing, CCA, Tx, turnaround, Rx, and sleep states. When MAC=False, nominal communication durations are used. When MAC=True, the simulated per-node CCA, Tx, waiting, and Rx durations replace the corresponding nominal values.

For PV-equipped nodes, harvested energy is calculated from the active PV voltage and current, the harvesting duration, and the MPP efficiency. The battery-lifetime estimate is idealized and does not account for aging, self-discharge, finite charge capacity, or nonlinear discharge behaviour.

10. Persistence and Reproducibility

Use `save_phy_state()` to store the detailed PHY matrices, `PhyResultsDTO.save_npz()` to store the compact PHY outputs required by the MAC and energy layers, and `EnergyManager.save_csv()` to export the final per-node results. For reproducibility, the design dictionary, configuration, software revision, MAC seeds, number of seeds, simulation duration, and PHY/MAC thresholds should be stored with these files.

When the geometry and Tx powers remain unchanged, the PHY results should be computed once and reused across different MAC seeds.

11. Minimal Execution Example

```
from enlight_iot.core.config import EnLightConfig
from enlight_iot.network.orchestrator import PhyNet
from enlight_iot.hardware.energy import EnergyManager

config = EnLightConfig()
phy = PhyNet(design, budget_run=False, config=config, btma_mode=True)
telemetry = phy.export_energy_telemetry()
energy = EnergyManager(telemetry, design, config=config,
                      MAC=True, btma_mode=True,
                      MAC_mode="unslotted")
results = energy.get_results_df()
energy.save_csv("experiment_results.csv")
```

12. Complete API Reference

This section summarises the public classes and functions, together with the internal methods that materially affect the scientific calculations. Dependencies, individual NumPy calls, state-assignment lists, and line-by-line implementation traces are omitted.

12.1 Module `channel.py`

Implements the optical and RF channel-gain calculations.

Class `ChannelEngine`

Uses the constructed Room and EnLightConfig to evaluate direct, diffuse, RIS-assisted, and RF links.

```
ChannelEngine(config: EnLightConfig, room: Room)
```

Method	Inputs	Purpose
<code>_get_blocker_params()</code>	None	Returns blocker positions, radius, and height. Internal helper.
<code>calc_h(tx, rx)</code>	Tx and Rx element batches	Calculates pairwise Lambertian optical gain before blockage.
<code>los_gains(tx, rx)</code>	Tx and Rx element batches	Calculates direct optical gain and sets blocked links to zero.
<code>diffuse_gains(tx, rx, bounces=4)</code>	Tx and Rx element batches; reflection order	Calculates multi-bounce diffuse gain through discretized room surfaces.
<code>ris_gains(tx, rx)</code>	Tx and Rx element batches	Calculates the two-hop Tx--RIS--Rx gain.
<code>rf_gains(tx, rx)</code>	Tx and Rx element batches	Calculates RF path loss using the configured indoor model.

- Optical matrices use pairwise Tx-by-Rx indexing.
- If no RIS is present, `ris_gains()` returns a zero matrix.
- RF parameters are read from `config.devices.rf_channel`.

12.2 Module `config.py`

Defines the dataclass-based defaults used throughout the library.

Class `PhysicsConfig`

Physical constants and unit vectors.

Class `SpectralConfig`

Wavelength range, spectral grid size, and solar temperature.

Class `EnvironmentConfig`

Room, reflectivity, receiver, RIS, and diffuse-reflection defaults.

Class `HardwareConfig`

MCU, power consumption, battery, and harvesting defaults.

Class `CommConfig`

Communication, data-rate, packet, task, cycle, and Rx-type defaults.

Class `DeviceConfig`

Dictionary-based parameters for the RF channel, TIA, PV circuit, and Tx drivers.

Class `EnLightConfig`

Master container grouping all configuration dataclasses.

```
EnLightConfig()
```

- The dataclasses contain fields only and define no explicit methods.
- Scenario-specific values are normally supplied through the design dictionary.

12.3 Module `devices.py`

Implements the receiver, transmitter driver, and PV circuit models.

Class `TIA`

Models TIA frequency response and integrated electrical noise.

```
TIA(config: EnLightConfig, **kwargs)
```

Method	Inputs	Purpose
CF(B)	Bandwidth	Calculates the feedback capacitance.
ZF(f, B)	Frequency and bandwidth	Calculates the frequency-dependent feedback impedance.
RF_psd(f, B)	Frequency and bandwidth	Calculates feedback-resistor thermal-noise PSD.
SV_psd(f, B)	Frequency and bandwidth	Calculates input-voltage-noise contribution.
SI_psd(f, B)	Frequency and bandwidth	Calculates input-current-noise contribution.
psd(f, B)	Frequency and bandwidth	Combines the TIA noise PSD components.
calc_noise_power(B, Nf=1000, fmin=0.1)	Receiver bandwidth	Numerically integrates the total TIA noise power.

Class `IRdriver`

Maps IR optical Tx power and driver current using fitted polynomials.

```
IRdriver(config: EnLightConfig, **kwargs)
```

Method	Inputs	Purpose
calc_I(P)	Optical Tx power	Returns the required driver current.
calc_P(I)	Driver current	Returns the corresponding optical Tx power.

Class `PV`

Calculates the PV DC operating point, bandwidth, transfer function, and noise.

```
PV(config, Gsignal, Gamb, unscaled=True, run=True, **kwargs)
```

Method	Inputs	Purpose
pv_current(V)	PV voltage	Evaluates the nonlinear PV current using the Lambert-W solution.
calc_capacitance()	None	Calculates the operating-point-dependent PV capacitance.
find_bw(verbose=False)	None	Calculates the small-signal PV bandwidth.
tf(f)	Frequency grid	Calculates the PV small-signal transfer function.
compute_all_noise(f)	Frequency grid	Calculates and integrates PV thermal-noise terms.
shot_noise(f)	Frequency grid	Calculates the integrated PV shot noise.
vp2p(f)	Frequency	Calculates the small-signal output voltage.

Module-level functions

Method	Inputs	Purpose
RF_calc_I(P, config, **kwargs)	RF Tx power and configuration	Estimates RF transmitter current consumption.

12.4 Module `energy.py`

Implements MAC-aware sensor-node energy, harvesting, and battery calculations.

Class `EnergyManager`

Combines the PHY telemetry, design energy profile, optional MAC simulation, harvesting model, and battery model.

```
EnergyManager(phy_data, design, config=None, MAC=False, btma_mode=False,  
MAC_mode='unslotted', **kwargs)
```

Method	Inputs	Purpose
_v(key, profile_sub=None)	Parameter name and optional design subsection	Resolves a value from the design and configuration hierarchy.
calc_cycle_energy()	None	Calculates per-state durations, currents, and energy per sensor cycle.
_apply_mac_times()	None	Runs the MAC layer and replaces nominal communication times with per-node simulated values.
calc_harv_energy()	None	Calculates PV harvesting power.
calc_battery_life()	None	Calculates daily energy balance and idealized battery lifetime.
get_results_df()	None	Returns the per-node PHY, MAC, energy, harvesting, and lifetime results.
save_csv(filepath)	Output path	Exports the result DataFrame to CSV.

- MAC simulation is invoked internally only when MAC=True.
- IR and RF MAC simulations are handled separately when both uplink types are present.

12.5 Module `geometry.py`

Defines batched spatial elements, room surfaces, and room construction.

Class `Elements`

Base container for batched 3D positions and orientation vectors.

```
Elements(r, n=[0,0,1])
```

Method	Inputs	Purpose
__add__(other)	Compatible element batch	Returns a combined element batch.
merge(batch_list)	List of compatible batches	Merges multiple batches into one container.

Class `OpticalRxElements`

Extends Elements with receiver area, field of view, reflectivity, and receiver type.

Class `OpticalTxElements`

Extends Elements with optical Tx power and Lambertian order.

Class `RFTxElements`

Extends Elements with RF Tx power.

Class `Surface`

Discretizes a rectangular wall, window, or RIS into optical elements.

```
Surface(config, center, dims, const_axis, resolution, ...)
```

Method	Inputs	Purpose
gen_surface_points(center, dims, const_axis, resolution)	Surface geometry	Returns the patch positions and patch area.

Class `RoomBuilder`

Parses the environment section of the design dictionary.

```
RoomBuilder(design, config, console=False)
```

Method	Inputs	Purpose
get_dimensions_and_res()	None	Resolves room dimensions and wall resolution.
get_reflectivity()	None	Resolves floor, ceiling, and wall reflectivities.
get_surfaces_by_type(surface_type)	Surface type	Collects windows or RIS definitions.

Class `Room`

Builds the six room boundaries and integrates windows, RISs, and blockers.

```
Room(room_builder, config, ignore_RIS=False, ignore_windows=False, console=False)
```

Method	Inputs	Purpose
_build_master_element()	None	Merges all room-wall element batches.
add_surface(new_surface)	Surface	Adds a window or RIS and removes overlapping wall patches.
plot_surface_addition(new_surface, overlap_mask)	Surface and overlap mask	Visualizes the retained and removed surface patches.

12.6 Module `interface.py`

Defines the compact data-transfer boundary between the PHY and energy/MAC layers.

Class `PhyResultsDTO`

Stores the PHY quantities required by EnergyManager without transferring the complete PHY state.

```
PhyResultsDTO(no_sensors, rb_up, rb_down, flag_pv, uplink_type, ...)
```

Method	Inputs	Purpose
save_npz(filepath)	Output path	Saves the telemetry to a compressed NumPy archive.
load_npz(filepath)	Input path	Loads a previously saved telemetry archive.

- The DTO includes PHY availability and hidden-node data required by the MAC layer.

12.7 Module `mac.py`

Implements the event-driven slotted and unslotted CSMA/CA model.

Class `MAC\_Params`

Stores MAC timing, traffic, frame, retry, PHY, and simulation parameters.

Method	Inputs	Purpose
symbol_duration_us()	None	Returns the symbol duration.
unit_backoff_us()	None	Returns the backoff-unit duration.
cca_duration_us()	None	Returns the CCA duration.
sifs_us()	None	Returns the SIFS duration.

Method	Inputs	Purpose
turnaround_us()	None	Returns the turnaround duration.
frame_bytes()	None	Returns the complete data-frame size.
frame_duration_us()	None	Returns the data-frame duration.
ack_duration_us()	None	Returns the ACK duration.

Class `SlotClock`

Provides a shared backoff-slot clock for slotted CSMA/CA.

Method	Inputs	Purpose
wait_slots(n)	Number of slots	Waits for the requested number of slot boundaries.

Class `VLC\_Channel`

Represents the shared channel, hidden-node relations, BTMA state, and collisions.

Method	Inputs	Purpose
busy()	None	Reports whether at least one transmission is active.
can_hear(listener, transmitter)	Node indices	Tests directed sensor-to-sensor visibility.
reaches_ap(node_id)	Node index	Tests whether the uplink reaches the AP.
cca(node_id)	Node index	Returns the CCA channel state perceived by the node.
start_tx(node_id)	Node index	Registers a transmission and marks collisions.
end_tx(node_id)	Node index	Ends a transmission and reports collision success.

Class `NodeStats`

Stores per-node packet, delay, collision, PHY-loss, and state-time counters.

Method	Inputs	Purpose
log_delay(gen_time, now)	Generation and completion times	Stores a successful packet delay.
mean_delay_us()	None	Returns mean successful delay.
p99_delay_us()	None	Returns the 99th-percentile successful delay.

Class `VLC\_Node`

Runs the packet source and CSMA/CA state machine for one node.

Method	Inputs	Purpose
_packet_source()	None	Generates periodic or Poisson traffic.
_queue_packet(gen_time)	Generation time	Serializes packet service at the node.
_csma_ca(gen_time)	Generation time	Executes backoff, CCA, Tx, ACK, and retry logic.

- VLC_Node methods are internal SimPy processes and use microseconds.

Module-level functions

Method	Inputs	Purpose
run_sim(...)	One complete MAC configuration	Runs one MAC realization and returns per-node statistics.
aggregate(node_stats, params)	One seed	Aggregates network-level metrics for one realization.
per_node_aggregate(all_seeds_stats, params)	Multiple seeds	Averages per-node statistics across seeds.

Method	Inputs	Purpose
call_MAC(...)	MAC configuration and seed count	Runs multiple seeds and returns mean, standard deviation, per-seed, and per-node results.
run_sweep(...)	Node-count sequence and MAC settings	Runs repeated MAC experiments over different network sizes.
plot_sweep(...)	Sweep results	Plots selected sweep metrics.

12.8 Module `nodes.py`

Parses node definitions and creates master, sensor, and ambient-node managers.

Class `NodeBuilder`

Extracts node parameters from the design dictionary and applies configuration fallbacks.

```
NodeBuilder(design, node_type, config, console=False)
```

Method	Inputs	Purpose
get_node_params(node_type)	Node-group name	Loads and normalizes the node parameters.
sanity_check()	None	Checks dimensions and separates IR and RF sensor properties.

Class `SNManager`

Creates sensor optical/RF elements, receiver masks, TIA model, and effective responsivities.

```
SNManager(node_builder, config)
```

Method	Inputs	Purpose
downlink_effective_responsivity()	None	Assigns the effective downlink responsivity for each receiver type and filter.

Class `MNManager`

Creates master node optical elements, TIA model, sensitivity, and uplink responsivities.

```
MNManager(node_builder, config)
```

Method	Inputs	Purpose
uplink_effective_responsivity()	None	Assigns the effective IR uplink responsivity.

Class `ANManager`

Creates ambient optical transmitters.

```
ANManager(node_builder)
```

12.9 Module `orchestrator.py`

Coordinates the complete physical-layer simulation.

Class `oPhyGains`

Stores and evaluates downlink, uplink, sensor-to-sensor, ambient, optical, and RF gains.

```
oPhyGains(room, masters, sensors, ambient, config, btma=True)
```

Method	Inputs	Purpose
compute_gains()	None	Calculates LoS, diffuse, RIS, RF, and optional sensor-to-sensor channel matrices.
compute_downlink()	None	Converts downlink gains into received optical powers and currents.

Method	Inputs	Purpose
compute_uplink()	None	Converts IR and RF uplink gains into received quantities.
compute_ambient()	None	Calculates artificial-light and window contributions.

Class `PhyNet`

Main PHY execution object.

```
PhyNet(design, budget_run=False, **kwargs)
```

Method	Inputs	Purpose
calc_min_ow_tx_power(target_ber)	Target BER	Calculates the minimum IR optical Tx power per sensor.
calc_min_rf_tx_power()	None	Calculates the minimum RF Tx power from path loss and sensitivity.
set_tx_power(target_ber=0.0038)	Target BER	Updates IR and RF sensor Tx powers for budget runs.
compute_noise()	None	Calculates PD/PV noise, receiver bandwidths, and PV operating quantities.
align_sensors_to_master()	None	Orients IR transmitters toward the single master node.
compute_metrics()	None	Calculates SNR, BER-related, link-availability, and component metrics.
save_phy_state(filepath)	Output path	Saves detailed PHY matrices.
export_energy_telemetry()	None	Returns a PhyResultsDTO for EnergyManager.

- Creating PhyNet executes the PHY workflow.
- Multiple master Tx-orientation optimization is not implemented for budget_run.

12.10 Module `physics.py`

Implements source spectra, detector responses, optical filters, and effective responsivity.

Class `SpectralPhysics`

Provides the spectral models used by the node and PV calculations.

```
SpectralPhysics(config: EnLightConfig)
```

Method	Inputs	Purpose
white_led_spectrum()	None	Returns the normalized white-LED spectrum.
tsff5210_spectrum()	None	Returns the normalized IR LED spectrum.
sun_spectrum()	None	Returns the normalized solar spectrum.
photodiode_responsivity()	None	Returns the photodiode responsivity curve.
solar_panel_sensitivity()	None	Returns the PV spectral sensitivity.
sun_power(l_min, l_max)	Wavelength interval	Calculates the solar power in the specified band.
get_filter_transmission(filter_name)	Filter name	Returns the selected optical-filter transmission.
calculate_effective_responsivity(source, detector, filter)	Spectral curves	Calculates the spectral-overlap effective responsivity.
get_responsivity_by_name(name)	Named source-detector-filter combination	Returns a predefined effective responsivity.

12.11 Module `utils.py`

Contains shared array, geometry, probability, and point-generation utilities.

Module-level functions

Method	Inputs	Purpose
to_vec_Nx3(N, arr)	Node count and vector input	Normalizes vectors to shape (N, 3).
to_scal_Nx1(N, arr)	Node count and scalar input	Normalizes values to shape (N, 1).
normalize_bool_array(arr, N)	Boolean input and node count	Normalizes Boolean masks.
as_array_of_size(value, N)	Scalar or array and node count	Broadcasts or validates a per-node array.
solar_panel_angular_efficiency(theta)	Incidence angle	Calculates PV angular response.
calculate_blockage_mask(tx, rx, blockers, radius, height)	Link endpoints and blocker geometry	Returns the pairwise LoS blockage mask.
generate_microgrids_vectorized(r, n, ...)	Patch positions and normals	Creates local microgrids for refined diffuse blockage.
Qfunction(x)	Input value	Evaluates the Gaussian Q-function.
Qinv(p)	Probability	Evaluates the inverse Gaussian Q-function.
generate_grid(...)	Room/grid parameters	Generates regular 2D or 3D point grids.
diagonal_points(...)	Endpoints and point count	Generates points along a diagonal.
align_to(source, target)	Source and target positions	Returns orientation vectors pointing toward the target.

Known Limitations and Maintenance Priorities

- Energy-layer MAC integration thresholds PHY SNR to link availability. A BER/PDR mapping would avoid an abrupt transition.
- Large diffuse and RIS matrices can dominate memory. Chunked or sparse formulations should be expanded.